

分布式计算实验报告

杨昕捷 22009201392

2025 年 6 月 5 日

1 题目一：MPI 使用梯形法计算连续函数积分

1.1 问题描述

实现基于 MPI 的并程序，使用梯形法（Trapezoidal Rule）计算连续函数 $f(x) = \sin(x^2 + x)$ 在区间 $[a, b]$ 上的定积分近似值。要求：

1. 使用 mpi4py 进行进程间通信
2. 均匀分割积分区间到各进程
3. 主进程（rank=0）汇总并输出最终结果

1.2 算法设计

1.2.1 梯形法原理

梯形法将积分区间 $[a, b]$ 划分为 n 个等宽子区间，每个子区间宽度为 $h = \frac{b-a}{n}$ 。积分近似公式为：

$$\int_a^b f(x)dx \approx \frac{h}{2} \left[f(a) + 2 \sum_{i=1}^{n-1} f(x_i) + f(b) \right]$$

其中 $x_i = a + i \cdot h$ 。

1.2.2 并行化策略

1. 将总区间 $[a, b]$ 均匀划分为 p 个子区间（ p 为进程数）
2. 每个进程 i 计算其子区间 $[a_i, b_i]$ 的局部积分
3. 通过 MPI_REDUCE 将局部积分汇总到主进程

1.3 实现细节

关键代码实现如下：

Listing 1: 并行梯形法实现

```
1 def trapezoidal_rule(a, b, n, func):  
2     h = (b - a) / n  
3     integral = (func(a) + func(b)) / 2.0  
4     for i in range(1, n):
```

```

5         x = a + i * h
6         integral += func(x)
7     return integral * h
8
9 def main():
10     comm = MPI.COMM_WORLD
11     rank = comm.Get_rank()
12     size = comm.Get_size()
13
14     # 参数设置
15     a, b, n = 0.0, 1.0, 1024 # 积分区间[0,10], 划分102400份
16
17     # 检查可整除性
18     # 只有整除才可以均分给所有的线程
19     if n % size != 0:
20         if rank == 0: print("Error: n must be divisible by size!")
21         return
22
23     local_n = n // size # 每个进程处理子区间数
24     h_total = (b - a) / n # 全局步长
25
26     # 计算本地积分区间
27     local_a = a + rank * local_n * h_total
28     local_b = local_a + local_n * h_total
29
30     # 本地积分计算 (使用向量化函数)
31     local_integral = trapezoidal_rule(
32         local_a, local_b, local_n,
33         lambda x: np.sin(x*x + x) # f(x)=sin(x^2+x)
34     )
35
36     # 结果归约到主进程
37     total_integral = comm.reduce(
38         local_integral,
39         op=MPI.SUM,
40         root=0
41     )
42
43     if rank == 0:
44         print(f"使用 {size} 个进程计算积分结果: {total_integral:.10f}")

```

1.4 执行结果

使用 4 个进程在区间 [0,1] 上计算结果:

```

(mpi) C:\Users\tingj\PycharmProjects\MPI>mpiexec -n 4 python src\TrapezoidalRule.py
使用 4 个进程计算积分结果: 0.6143218007

```

图 1: TrapezoidalRuleResult

1.5 结果分析

- 负载均衡：各进程处理相同的子区间长度 $\frac{b-a}{p}$
- 边界处理：梯形法要求子区间边界点被相邻进程共享计算，本实现通过精确划分保证边界值无重复计算

2 题目二：Hadoop 平台实现 MapReduce 统计成绩

2.1 问题描述

输入文件为学生成绩信息，包含了必修课与选修课成绩，格式如下：

班级 1, 姓名 1, 科目 1, 必修, 成绩 1
 （注：
 为换行符）

班级 2, 姓名 2, 科目 1, 必修, 成绩 2

班级 1, 姓名 1, 科目 2, 选修, 成绩 3

.....,,,

编写 Hadoop 平台上的 MapReduce 程序，分别实现如下功能：

1. 计算每个学生必修课的平均成绩。
2. 统计每个班级中所有课程（必修 + 选修）平均成绩排名前五的学生姓名和成绩。

2.1.1 并行化策略

使用两个 MapReduce 程序分别实现问题，对于第一个问题，针对输入字符串，Map 成 < 姓名, 成绩 >，注意在进行 Map 时会同时将选修课成绩去除；接着在 Reduce 阶段，通过姓名进行分组，以实现问题的并行化处理。

针对第二个问题，则使用两个 MapReduce 进行解决，第一个 MapReduce 同问题一，得到每个学生的平均成绩，Map 成 < (班级名, 姓名), 成绩 >，Reduce 输出 < (班级名, 姓名), 平均成绩 >。第二个 MapReduce 则根据班级进行 Map 分类，把每一个班级的学生聚集到同一个类中，这样就可以在 Reduce 时进行排名，通过维护一个大小为 5 的最小堆进行实现。

2.2 实现细节

关键代码实现如下：

Listing 2: 计算必修课平均成绩实现

```
1 package com.org.xidian;
2 import org.apache.hadoop.conf.Configuration;
3 import org.apache.hadoop.fs.Path;
4 import org.apache.hadoop.io.DoubleWritable;
5 import org.apache.hadoop.io.LongWritable;
6 import org.apache.hadoop.io.Text;
7 import org.apache.hadoop.mapreduce.Job;
8 import org.apache.hadoop.mapreduce.Mapper;
9 import org.apache.hadoop.mapreduce.Reducer;
10 import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
11 import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
```

```

12
13 import java.io.IOException;
14
15 public class MapReduceScoreAverage {
16     //自定义的mapper, 继承org.apache.hadoop.mapreduce.Mapper
17     public static class MyMapper extends org.apache.hadoop.mapreduce.Mapper<
18         LongWritable, Text, Text, LongWritable> {
19         @Override
20         protected void map(LongWritable key, Text value, Mapper<LongWritable, Text,
21             Text, LongWritable>.Context context)
22             throws IOException, InterruptedException {
23             String line = value.toString();
24             // split函数是用于按指定字符(串)或正则去分割某个字符串, 结果以字符串
25             // 数组形式返回, 这里按照"\t"来分割text文件中字
26             // 符, 即一个制表符, 这就是为什么我在文本中用了空格分割, 导致最后的结果
27             // 有很大的出入。
28             // input format : "班级 1, 姓名 1, 科目 1, 必修, 成绩 1 <br>"
29             String[] splitWords = line.split(",");
30
31             // filter
32             String lessonType = splitWords[3];
33             if(!lessonType.equals("必修")) return;
34
35             // parse data from splitWords
36             String studentName = splitWords[1];
37             long score = Long.parseLong(splitWords[4]);
38
39             // output format : <"studentName",score>
40             context.write(new Text(studentName), new LongWritable(score));
41         }
42     }
43
44     public static class MyReducer extends org.apache.hadoop.mapreduce.Reducer<Text,
45         LongWritable, Text, DoubleWritable> {
46         @Override
47         protected void reduce(Text k2, Iterable<LongWritable> v2s,
48             Reducer<Text, LongWritable, Text, DoubleWritable>.
49                 Context context)
50             throws IOException, InterruptedException {
51
52             long count = 0;
53             long sum = 0;
54             for (LongWritable v2 : v2s) {
55                 count++;
56                 sum += v2.get();
57             }
58             DoubleWritable v3 = new DoubleWritable((double) sum / count);
59             context.write(k2, v3);
60         }
61     }
62 }

```

```

55     // after the tasks finish, print the result
56     @Override
57     protected void cleanup(Context context) {
58         // 这里做一些全局统计, 比如打印到日志
59         System.out.println("Done");
60     }
61
62 }
63
64 // 客户端代码, 写完交给ResourceManager框架去执行
65 public static void main(String[] args) throws Exception {
66     Configuration conf = new Configuration();
67     Job job = Job.getInstance(conf, MapReduceScoreAverage.class.getSimpleName()
68         );
69     // 打成jar执行
70     job.setJarByClass(MapReduceScoreAverage.class);
71
72     // 数据在哪里?
73     FileInputFormat.setInputPaths(job, args[0]);
74     // 使用哪个mapper处理输入的数据?
75     job.setMapperClass(MyMapper.class);
76     // map输出的数据类型是什么?
77     job.setMapOutputKeyClass(Text.class);
78     job.setMapOutputValueClass(LongWritable.class);
79
80     // 使用哪个reducer处理输入的数据?
81     job.setReducerClass(MyReducer.class);
82     // reduce输出的数据类型是什么?
83     job.setOutputKeyClass(Text.class);
84     job.setOutputValueClass(DoubleWritable.class);
85     // 数据输出到哪里?
86     FileOutputFormat.setOutputPath(job, new Path(args[1]));
87
88     // 交给yarn去执行, 直到执行结束才退出本程序
89     job.waitForCompletion(true);
90 }

```

Listing 3: 计算班级平均成绩最高的五人

```

1 package com.org.xidian;
2
3 import org.apache.hadoop.conf.Configuration;
4 import org.apache.hadoop.fs.Path;
5 import org.apache.hadoop.io.DoubleWritable;
6 import org.apache.hadoop.io.LongWritable;
7 import org.apache.hadoop.io.Text;
8 import org.apache.hadoop.mapreduce.Job;
9 import org.apache.hadoop.mapreduce.Mapper;
10 import org.apache.hadoop.mapreduce.Reducer;

```

```

11 import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
12 import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
13
14 import java.io.IOException;
15 import java.util.*;
16
17 public class MapReduceSortByClass {
18     //自定义的mapper, 继承org.apache.hadoop.mapreduce.Mapper
19     public static class MyMapper1 extends Mapper<LongWritable, Text, Text,
20         LongWritable> {
21         @Override
22         protected void map(LongWritable key, Text value, Mapper<LongWritable, Text,
23             Text, LongWritable>.Context context)
24             throws IOException, InterruptedException {
25             String line = value.toString();
26             // input format : "班级 1, 姓名 1, 科目 1, 必修, 成绩 1 <br>"
27             // split by ','
28             String[] splitWords = line.split(",");
29
30             // no filter because sorting according to scores of all lessons
31             // String lessonType = splitWords[3];
32             // if(!lessonType.equals("必修")) return;
33
34             // parse data from splitWords
35             String className = splitWords[0];
36             String studentName = splitWords[1];
37             long score = Long.parseLong(splitWords[4]);
38
39             // output format : <"studentName",score>
40             context.write(new Text(className + ',' + studentName), new LongWritable
41                 (score));
42         }
43     }
44
45     public static class MyReducer1 extends Reducer<Text, LongWritable, Text,
46         DoubleWritable> {
47         @Override
48         protected void reduce(Text k2, Iterable<LongWritable> v2s,
49             Reducer<Text, LongWritable, Text, DoubleWritable>.
50                 Context context)
51             throws IOException, InterruptedException {
52
53             long count = 0;
54             long sum = 0;
55             for (LongWritable v2 : v2s) {
56                 count++;
57                 sum += v2.get();
58             }
59             DoubleWritable v3 = new DoubleWritable((double) sum / count);

```

```

55         context.write(k2, v3);
56     }
57
58     // after the tasks finish, print the result
59     @Override
60     protected void cleanup(Context context) {
61         System.out.println("Job 1 Done");
62     }
63
64 }
65
66 public static class MyMapper2 extends Mapper<LongWritable, Text, Text, Text> {
67     @Override
68     protected void map(LongWritable key, Text value, Mapper<LongWritable, Text,
69         Text, Text>.Context context)
70         throws IOException, InterruptedException {
71         String line = value.toString();
72         // input format : "班级 1, 姓名 1, 平均成绩 1 <br>"
73         // split by ','
74         String[] splitWords = line.split(",");
75
76         // parse data from splitWords
77         String className = splitWords[0];
78         String studentName = splitWords[1];
79         String score = splitWords[2];
80
81         // output format :<"className", "student,score">
82         context.write(new Text(className), new Text(studentName + ',' + score))
83         ;
84     }
85 }
86
87 public static class MyReducer2 extends Reducer<Text, Text, Text, DoubleWritable
88     > {
89     @Override
90     protected void reduce(Text k2, Iterable<Text> v2s,
91         Reducer<Text, Text, Text, DoubleWritable>.Context
92         context)
93         throws IOException, InterruptedException {
94
95         // create a heap to store the top 5
96         TopTracker top5Tracker = new TopTracker(5);
97
98         for (Text v2 : v2s) {
99             // extract name and score from text
100             String[] splitWords = v2.toString().split(",");
101             String studentName = splitWords[0];
102             double score = Double.parseDouble(splitWords[1]);
103             // add it into the heap to replace the last rank of top 5

```

```

100         top5Tracker.add(studentName, score);
101     }
102
103     // loop to output top 5
104     for (Map.Entry<String, Double> entry : top5Tracker.topK()) {
105         DoubleWritable v3 = new DoubleWritable(entry.getValue());
106         context.write(new Text(entry.getKey()), v3);
107     }
108 }
109
110 // after the tasks finish, print the result
111 @Override
112 protected void cleanup(Context context) {
113     System.out.println("Job 2 Done");
114 }
115
116 /*
117 A private class using for track top k
118 */
119 private static class TopTracker {
120     private final PriorityQueue<Map.Entry<String, Double>> minHeap;
121     private final int capacity;
122
123     public TopTracker(int k) {
124         this.capacity = k;
125         // 创建一个基于值的最小堆
126         this.minHeap = new PriorityQueue<>(
127             k, Comparator.comparing(Map.Entry::getValue)
128         );
129     }
130
131     /**
132      * 尝试添加新键值对 (name, score)
133      */
134     public void add(String name, double score) {
135         Map.Entry<String, Double> entry = Map.entry(name, score);
136
137         if (minHeap.size() < capacity) {
138             minHeap.offer(entry);
139         } else {
140             assert minHeap.peek() != null;
141             if (score > minHeap.peek().getValue()) {
142                 minHeap.poll();
143                 minHeap.offer(entry);
144             }
145         }
146     }
147
148     /**

```



```

149         * 返回 Top-K, 按分数降序排列
150         */
151         public List<Map.Entry<String, Double>> topK() {
152             List<Map.Entry<String, Double>> result = new ArrayList<>(minHeap);
153             result.sort(Comparator.comparing(Map.Entry::getValue));
154             Collections.reverse(result);
155             return result;
156         }
157     }
158 }
159
160
161 // 客户端代码, 写完交给ResourceManager框架去执行
162 public static void main(String[] args) throws Exception {
163     Configuration conf = new Configuration();
164     // set default split between key and value to ','
165     conf.set("mapreduce.output.textoutputformat.separator", ",");
166
167     // Job1
168     Job job1 = Job.getInstance(conf, "Phase1");
169     job1.setJarByClass(MapReduceSortByClass.class);
170     FileInputFormat.setInputPaths(job1, args[0]);
171     job1.setMapperClass(MyMapper1.class);
172     job1.setMapOutputKeyClass(Text.class);
173     job1.setMapOutputValueClass(LongWritable.class);
174     job1.setReducerClass(MyReducer1.class);
175     job1.setOutputKeyClass(Text.class);
176     job1.setOutputValueClass(DoubleWritable.class);
177     FileOutputFormat.setOutputPath(job1, new Path(args[1]));
178
179     // if job1 not finish, wait
180     boolean isSuccess = job1.waitForCompletion(true);
181     if (!isSuccess) {
182         System.err.println("Job1 failed, exiting");
183         System.exit(1);
184     }
185
186     // Job2
187     Job job2 = Job.getInstance(conf, "Phase2");
188     job2.setJarByClass(MapReduceSortByClass.class);
189     FileInputFormat.setInputPaths(job2, args[1]);
190     job2.setMapperClass(MyMapper2.class);
191     job2.setMapOutputKeyClass(Text.class);
192     job2.setMapOutputValueClass(Text.class);
193     job2.setReducerClass(MyReducer2.class);
194     job2.setOutputKeyClass(Text.class);
195     job2.setOutputValueClass(DoubleWritable.class);
196     FileOutputFormat.setOutputPath(job2, new Path(args[2]));
197
198     boolean isSuccess2 = job2.waitForCompletion(true);

```

```

198     if (!isSuccess2) {
199         System.err.println("Job2 failed, exiting");
200         System.exit(1);
201     }
202
203 }
204 }

```

2.3 执行结果

```

龚康      84.57142857142857
龚彤欣    72.0
龚志      68.14285714285714
龚志兵    86.28571428571429
龚思      78.57142857142857
龚怡      79.57142857142857
龚敦豪    67.14285714285714
龚文      71.0
龚昌小    80.0
龚昌轩    69.71428571428571
龚显      81.28571428571429
龚显杰    80.71428571428571
龚杰伟    79.57142857142857
龚析      76.0
龚正晰    70.71428571428571
龚民兵    70.57142857142857
龚泰      62.285714285714285
龚泰民    69.42857142857143
龚清和    73.85714285714286
龚琪山    83.57142857142857
龚瑶娟    79.0
龚盈      68.57142857142857
龚睿      67.14285714285714
龚笑欣    74.0
龚维      79.57142857142857
龚耀      71.0
龚耀显    80.42857142857143
龚良乎    70.28571428571429

```

图 2: MapReduceScoreAverageResult

```

sandbox@clientnode:~/ScoreAnalyse$ hadoop fs -cat output/part-r-00000
韩琪,90.125
常丽杰,88.875
潘英义,87.14285714285714
梁帅,87.125
宋睿,86.88888888888889
魏和伦,89.3
江勤颖,88.57142857142857
崔盈,88.0
黎清,87.75
孔耀健,87.375
汪贤,90.44444444444444
魏威晰,89.875
傅丽浩,89.5
汤丽灵,89.5
郑志,88.125
秦勤仁,86.85714285714286
胡和,86.44444444444444
崔诚仁,86.11111111111111
宋伦昌,85.75
邵健军,85.6
林华华,90.0
蒋晰,88.55555555555556
赵石宁,87.75
段媚义,87.66666666666667
叶姿思,87.11111111111111
阎宇英,88.0

```

图 3: MapReduceSortByClassResult

2.4 注意事项

- 必须有“`job.waitForCompletion(true)`”，否则 MapReduce 任务不会开始。
- 在开始任务之前，必须先把相关数据通过“`hadoop fs -put ./grades.txt input`”指令进行上传
- MapReduce 结果的默认分隔符是制表符，多步 MapReduce 有可能会因为 split 时分隔符不统一而出现问题，在第二个问题中使用了“`conf.set("mapreduce.output.textoutputformat.separator", ",")`”将默认的制表符设置成了“`,`”。

3 题目三：Hadoop 平台实现 MapReduce 找到祖孙关系

3.1 问题描述

输入文件的每一行为具有父子/父女/母子/母女/关系的一对人名，例如：

Tim, Andy

Harry, Alice

Mark, Louis

Andy, Joseph

.....,

假定不会出现重名现象。

编写 Hadoop 平台上的 MapReduce 程序，找出所有具有 grandchild-grandparent 关系的人名对。

3.1.1 问题思路

在这个问题中，核心思想是通过一个中间人串联起祖孙关系。在 Map 阶段将一对关系拆成两对关系，比如说对于 Tim, Andy，就可以知道 Tim's Parent 是 Andy，反过来 Andy's child 是 Tim，这样就从初始的一组拆成了，`<Andy,"C", Tim">` 和 `<Tim,"P", Andy">`。

接着 Reduce 则根据中间人，即每一组的 Key 进行分类，在其中有一组是 Parent，一组是 Child，通过求这两者的笛卡尔积就可以得到所有满足关系的对。

3.2 实现细节

关键代码实现如下：

Listing 4: 计算祖孙关系实现

```
1 package com.org.xidian;
2
3 import org.apache.hadoop.conf.Configuration;
4 import org.apache.hadoop.fs.Path;
5 import org.apache.hadoop.io.LongWritable;
6 import org.apache.hadoop.io.Text;
7 import org.apache.hadoop.mapreduce.Job;
8 import org.apache.hadoop.mapreduce.Mapper;
9 import org.apache.hadoop.mapreduce.Reducer;
10 import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
11 import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
12
```

```

13 import java.io.IOException;
14 import java.util.ArrayList;
15 import java.util.List;
16
17 public class MapReduceFindGrand {
18     //自定义的mapper, 继承org.apache.hadoop.mapreduce.Mapper
19     public static class MyMapper extends org.apache.hadoop.mapreduce.Mapper<
20         LongWritable, Text, Text, Text> {
21         @Override
22         protected void map(LongWritable key, Text value, Mapper<LongWritable, Text,
23             Text, Text>.Context context)
24             throws IOException, InterruptedException {
25             String line = value.toString();
26             String[] splitWords = line.split(",");
27             String child = splitWords[0];
28             String parent = splitWords[1];
29             context.write(new Text(child), new Text("P,"+parent));
30             context.write(new Text(parent), new Text("C,"+child));
31         }
32     }
33
34     public static class MyReducer extends org.apache.hadoop.mapreduce.Reducer<Text,
35         Text, Text, Text> {
36         @Override
37         protected void reduce(Text k2, Iterable<Text> v2s,
38             Reducer<Text, Text, Text, Text>.Context context)
39             throws IOException, InterruptedException {
40             List<String> parents = new ArrayList<>();
41             List<String> children = new ArrayList<>();
42             for (Text v2 : v2s) {
43                 String[] splitWords = v2.toString().split(",");
44                 // if v2 is k2's parent then
45                 if(splitWords[0].equals("P")){
46                     parents.add(splitWords[1]);
47                 }else {
48                     children.add(splitWords[1]);
49                 }
50             }
51             // get the Cartesian product of these two lists and output
52             for (String parent:parents){
53                 for (String child:children){
54                     context.write(new Text(child),new Text(parent));
55                 }
56             }
57         }
58     }
59
60     //客户端代码, 写完交给ResourceManager框架去执行
61     public static void main(String[] args) throws Exception {

```

```

58     Configuration conf = new Configuration();
59     Job job = Job.getInstance(conf, MapReduceFindGrand.class.getSimpleName());
60     //打成jar执行
61     job.setJarByClass(MapReduceFindGrand.class);
62
63     //数据在哪里?
64     FileInputFormat.setInputPaths(job, args[0]);
65     //使用哪个mapper处理输入的数据?
66     job.setMapperClass(MyMapper.class);
67     //map输出的数据类型是什么?
68     job.setMapOutputKeyClass(Text.class);
69     job.setMapOutputValueClass(Text.class);
70
71     //使用哪个reducer处理输入的数据?
72     job.setReducerClass(MyReducer.class);
73     //reduce输出的数据类型是什么?
74     job.setOutputKeyClass(Text.class);
75     job.setOutputValueClass(Text.class);
76     //数据输出到哪里?
77     FileOutputFormat.setOutputPath(job, new Path(args[1]));
78
79     //交给yarn去执行,直到执行结束才退出本程序
80     job.waitForCompletion(true);
81 }
82 }

```

3.3 执行结果

```

Fred      Emma
Daisy     Bob
Tanya     Greta
Angelia   Greta
Jill      Robinson
Noah      Robinson
Jerry     Robinson
Heidi     Brown
Hank      Brown
Katherine Brown
Kimberly  Brown
Elvis     Carrie
Tina      Carrie
Andrew    Carrie
Jenna     Josephine
Rita      Josephine
Nathaniel Josephine
Miranda   Josephine
Natalie   Leslie
Bruce     Leslie
Hunk      Leslie
Darcy     Geoffrey
Christy   Geoffrey
Janet     Geoffrey
sandbox@clientnode:~/FindGrant$

```

图 4: MapReduceFindGrandResult

4 题目四：Hadoop 平台实现 MapReduce 找到祖孙关系

4.1 问题描述

输入文件为学生成绩信息，包含了必修课与选修课成绩，格式如下：

班级 1, 姓名 1, 科目 1, 必修, 成绩 1
 （注：
 为换行符）

班级 2, 姓名 2, 科目 1, 必修, 成绩 2

班级 1, 姓名 1, 科目 2, 选修, 成绩 3

.....,,,

编写 Spark 程序，实现如下功能：

按班级统计必修课平均成绩在：90 100、80 89、70 79、60 69 和 60 分以下这 5 个分数段的学生人数。

4.1.1 问题思路

这个问题类似于题目二中的多次 MapReduce 实现，只不过这里使用 spark 实现，使用 python 减少了代码的复杂度。对于一些 MapReduce 过程可以直接调用方法或是传入 lambda 匿名函数。

4.2 实现细节

关键代码实现如下：

Listing 5: 统计学生人数

```
1 from pyspark import SparkConf, SparkContext
2 import sys
3
4 # Usage: spark-submit ScoreClassify.py <input_path> <output_path>
5 conf = SparkConf().setAppName("RequiredCourseStats")
6 sc = SparkContext(conf=conf)
7
8 # Paths from command-line
9 if len(sys.argv) != 3:
10     print("Usage: spark-submit grade_stats.py <input_path> <output_path>")
11     sys.exit(1)
12 input_path = sys.argv[1]
13 output_path = sys.argv[2]
14
15 # Read input file
16 lines = sc.textFile(input_path)
17
18 # Parse each line: 班级, 姓名, 科目, 必修/选修, 成绩
19 def parse_line(line):
20     parts = [p.strip() for p in line.split(',')]
21     if len(parts) != 5:
22         return None
23     clazz, name, subject, ctype, score_str = parts
24     try:
25         score = float(score_str)
26     except ValueError:
```

```

27         return None
28     return clazz, name, ctype, score
29
30 parsed = lines.map(parse_line).filter(lambda x: x is not None)
31
32 # Filter required courses only
33 required = parsed.filter(lambda x: x[2] == '必修')
34
35 # Map to ((class, student), score)
36 stu_scores = required.map(lambda x: ((x[0], x[1]), x[3]))
37
38 # Compute average score per student
39 avg_scores = stu_scores.groupByKey() \
40     .mapValues(lambda scores: sum(scores) / len(scores))
41
42 # Assign score into buckets
43 def score_bucket(avg):
44     if avg >= 90:
45         return '90-100'
46     elif avg >= 80:
47         return '80-89'
48     elif avg >= 70:
49         return '70-79'
50     elif avg >= 60:
51         return '60-69'
52     else:
53         return '60以下'
54
55 # Map to ((class, bucket), 1) and count students
56 class_bucket = avg_scores.map(
57     lambda x: ((x[0][0], score_bucket(x[1])), 1)
58 )
59
60 counts = class_bucket.reduceByKey(lambda a, b: a + b)
61
62 # Format result as CSV: class,bucket,count
63 output = counts.map(lambda x: f"{x[0][0]},{x[0][1]},{x[1]}")
64
65 # Save to output
66 output.saveAsTextFile(output_path)
67
68 sc.stop()

```

4.3 执行结果

```
sandbox@clientnode:~$ hadoop fs -cat output/part-00000
160312班,70-79,499
170314班,70-79,477
170313班,80-89,136
170313班,70-79,480
170312班,60-69,157
160313班,70-79,493
160315班,70-79,461
170312班,70-79,431
170314班,80-89,136
160314班,70-79,479
170312班,80-89,118
160312班,60-69,163
170314班,60-69,137
170313班,60-69,149
160312班,80-89,134
160314班,80-89,124
160313班,60-69,168
160314班,60-69,176
160315班,60-69,149
160313班,80-89,127
160315班,80-89,102
170312班,60以下,3
170314班,60以下,5
160314班,60以下,4
160313班,60以下,3
160301班,90-100,2
160312班,60以下,1
160315班,60以下,2
170301班,90-100,2
170313班,60以下,1
170315班,90-100,2
```

图 5: ScoreClassifyResult

5 实验体会

首先实验环境的搭建展示了使用 docker 技术对于分布式计算的节点部署方式进行了大量的化简,并且公用 linux 内核的设计思想使得使用较小的性能开销搭建灵活的测试与服务平台。但是 Docker 自带的 GUI 并不好用,更好的方式是使用 Docker-Compose 直接进行部署,并且使用命令行和配置文件可以使部署更加规范。

本次实验通过 MPI、MapReduce 和 Spark 三大分布式框架的实践,深刻揭示了不同并行计算范式的应用方式与思想。在 MPI 的灵活进程控制中,我体会到手动优化通信对计算密集型任务的关键性;MapReduce 的两阶段分治设计,可以直观的对于任务进行拆分,将多作业串联的复杂度也暴露了其迭代场景的局限而 Spark 的内存计算与高阶抽象,则以简洁的 API 实现高效迭代。

更重要的是,实验让我领悟到分布式系统的本质:通过对于任务的拆分,让原本需要顺序计算的资源变成了能够充分运用处理器和集群多核心优势的计算架构。同时也让我了解了当下常用的分布式计算架构,使我受益匪浅,这会在日后的工作中成为相当有用的助力。